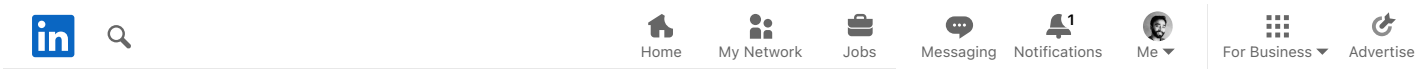
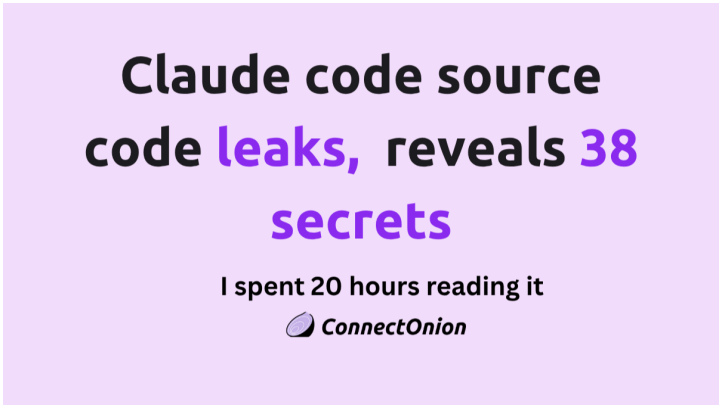

Home
My Network
Jobs
Messaging
Notifications
1
Me
For Business
Advertise



Claude code source code leaks, reveals 38 secrets

Aaron xie
open source AI agent (ConnectOnion) /CEO of OpenOnion/ machine learning engineer/Blockchain Engineer/sci-fi...

March 31, 2026

In March 2026, Claude Code's source got dug out. I built a site to analyze it in depth.

cc.openonion.ai

The previous article analyzed Claude Code's context injection mechanism. This one is about the details. The source contains a lot of design decisions. Some are elegant. Some are hilarious. Some make you rethink what an "AI tool" actually is.

---

1. Pets

Source location: src/buddy/

Open Claude Code and you'll see a little robot next to the input box. Most people assume it's a static icon.

In the code it's called "Buddy," but the user-facing name is Kettle.

There are 18 species to choose from: duck, goose, cat, dragon, octopus, owl, penguin, turtle, snail, ghost, axolotl, capybara, cactus, robot, rabbit, mushroom, and one called "chonk," a spherical creature.

Each species has hand-drawn ASCII art and three animation frames. The duck looks like this:

```
    __
  <(· )___
   (  ._>
    `--´
```

The cat:

```
/\_/\
   ( ·   ·)
   (  ω  )
   (")_(")
```

{E} is an eye placeholder. At runtime it gets replaced with one of six eye styles: ·, ✦, ×, ◉, @, °. Which eyes your pet gets depends on your user ID.

Rarity System

Each user's pet is determined by a hash of the user ID using the Mulberry32 pseudo-random generator. It's deterministic: the same user always gets the same pet.

Rarity has five tiers:

Five stats: DEBUGGING, PATIENCE, CHAOS, WISDOM, SNARK.

Legendary pets display five stars ★★★★★, and there's also a 1% chance of a "shiny" variant.

Eight hats: none, crown, top hat, propeller hat, halo, wizard hat, beanie, and a little duck hat. A robot wearing a duck hat.

Anti-Cheat Design

The pet's "bones" - species, rarity, eyes, hat - are regenerated from the userId hash every time. Editing the config file doesn't help; the system recomputes everything from your ID.

But the pet's "soul" - name and personality - is generated only once when it first "hatches," then stored in config. The bones can't be forged. The soul can't be swapped.

The salt value is friend-2026-401. April 1, 2026. April Fool's Day.

---

2. Opus 4.6 Fast Mode Costs 6x More Than Standard Mode

Source location: src/utils/modelCost.ts

The constants in the source are blunter than the pricing page:

```
Sonnet:        $3 input / $15 output (per million
tokens)
Opus 4/4.1:    $15 input / $75 output
Opus 4.5/4.6:  $5 input / $25 output
Opus 4.6 Fast: $30 input / $150 output
Haiku 3.5:     $0.80 input / $4 output
Haiku 4.5:     $1 input / $5 output
```

Fast mode is priced at 6x standard Opus 4.6. When you switch with /

fast, the system prompt tells you it "uses the same model, just outputs faster." That's true. The model doesn't change. The price does. Faster inference takes more compute, and Anthropic charges 6x for it.

Another detail: Prompt Cache reads cost 10% of writes, and writes cost 25% of input price. Web Search costs $0.01 per call. None of that is shown in the UI, but it's written plainly in the source.

---

3. A Full Vim State Machine

Source location: src/vim/

Claude Code has a full Vim mode. A real finite state machine.

```
INSERT <-> NORMAL

From NORMAL idle state you can enter:
  d/c/y -> operator (delete/change/yank)
  1-9   -> count (numeric prefix)
  f/F/t/T -> find (character search)
  g     -> g-command
  r     -> replace
  >/<   -> indent
```

Operator combinations are supported: d2w deletes two words, 2d5w deletes five words twice. Text objects are supported too: diw deletes inner word, da(deletes including the parentheses.

Persistent state carries across commands: last search, last operation, paste buffer.

Why does an AI coding tool need Vim mode? Because Claude Code users are programmers. Programmers use Vim. If you're editing a long prompt in the terminal, you want 0 to jump to the beginning of the line, dw to delete a word, and f" to jump to the next quote.

This is knowing who your user is.

---

4. Voice Mode Has a Switch Called tengu_amber_quartz

Source location: src/voice/voiceModeEnabled.ts

The voice mode code is short, but dense.

Conditions for enabling it:

1. It must be an Anthropic OAuth login. API Key, Bedrock, Vertex, and Foundry do not qualify.
2. The GrowthBook feature flag VOICE_MODE must be on.
3. Another flag, tengu_amber_quartz_disabled, must be off.

The third flag is a kill switch. The name has no semantic meaning at all: tengu plus amber_quartz. That's deliberate. If feature flags had names like voice_mode_kill_switch, anyone reading the source could find them and guess what they control or when they trigger. A meaningless name

and guess what they control or when they trigger. A meaningless name turns the flag into a black box. Only the internal team knows what it does.

There are dozens of similar flags in the source, all starting with tengu_: tengu_glacier_2xr, tengu_ccr_bridge_multi_session, tengu_scratch, tengu_tool_pear. Tengu is a creature from Japanese folklore. Anthropic uses it as the prefix for feature flags.

Voice mode uses claude.ai's voice_stream endpoint. On macOS, the first permission check calls the system security command, taking 20-50 milliseconds, then caches the result. Tokens refresh once per hour.

---

5. Coordinator Mode: One AI Managing a Group of AIs

Source location: src/coordinator/coordinatorMode.ts

Set the environment variable CLAUDE_CODE_COORDINATOR_MODE=1, and Claude Code becomes coordinator mode.

The coordinator doesn't write code directly. It manages a group of workers, assigns tasks, synthesizes results, and reports back to the user.

More than 370 lines of system prompt define the coordinator's behavior:

```
You are Claude Code, an AI assistant that
orchestrates
software engineering tasks across multiple workers.
```

Key rules for the coordinator:

- If it can answer the question itself, don't delegate to a worker.
- Worker results are internal signals. Don't thank them.
- Research tasks can run in parallel. Implementation tasks should run sequentially.

Workers have two modes. Simple mode gets only Bash, Read, and Edit. Full mode gets 30+ standard tools plus Skills.

There are internal tools the user never sees: TeamCreateTool, TeamDeleteTool, SendMessageTool, SyntheticOutputTool. When a worker finishes, the system sends its status, result, and token usage back to the coordinator with an XML <task-notification> tag.

The default maximum is 32 parallel sessions.

There's also a feature called scratchpad, a shared knowledge directory across workers, controlled by the tengu_scratch feature flag. Worker A can discover something, write it into the scratchpad, and Worker B can read it later.

---

6. Tool Lazy Loading

Claude Code has 40+ tools.

In src/Tool.ts, each tool has two key properties:

```
shouldDefer?: boolean   // requires ToolSearch before
use
alwaysLoad?: boolean    // always load in the first
round
```

MCP (Model Context Protocol) tools are all deferred by default. If the user installs 10 MCP servers and each one exposes 5 tools, that's 50 tools. Put all of them into the system prompt and token cost explodes.

The solution is the ToolSearch tool. The model doesn't know the full parameter definitions of deferred tools. It only knows their names. When it needs one, it calls ToolSearch to retrieve the full definition, and only then can it use it.

A few tools are never deferred:

- ToolSearch itself, because the loader can't be lazily loaded
- Agent tools
- Brief, the communication channel
- SendUserFile

This is an interesting engineering decision. The length of the system prompt directly affects both cost and speed. Lazy loading compresses the tool list down to names and expands it only when needed. The price is one extra API round trip.

---

7. Native Client Attestation: Placeholder Overwrite

Source location: src/constants/system.ts

Claude Code has an authentication mechanism in its HTTP headers:

```
const cch = feature('NATIVE_CLIENT_ATTESTATION') ? '
cch=00000;' : ''
```

cch=00000 is a placeholder. In the serialized request body, those five zeroes are overwritten at send time by Bun's HTTP stack with the computed attestation token.

Why not just fill in the correct value while building the request? Because the attestation token depends on the contents of the request body. Serialize first, compute token second, then write it back, and you end up reallocating memory. Reserve a fixed-length placeholder and overwrite it in place, and you get zero extra allocations.

This is system-level programming thinking. Most web developers wouldn't care about one extra allocation during JSON serialization. But Claude Code might send dozens of requests per second. At that scale, it adds up.

8. Every Tool Has a Security Classification Card

src/Tool.ts doesn't define tools as just "name + parameters + execution function." Each tool carries a set of security metadata:

```
isConcurrencySafe()     // can it run in parallel
isReadOnly()            // is it read-only
isDestructive()         // is it irreversible
interruptBehavior()     // on interrupt: cancel or
block
isSearchOrReadCommand() // search, read, or list
isOpenWorld()           // does it involve an
external system
```

There's also a method called toAutoClassifierInput() that compresses a tool call into a compact one-line representation and feeds it to the safety classifier. For example, a Bash call like ls -la gets compressed to that literal string, and the classifier decides whether it's safe.

Tools also have a permission check:

```
checkPermissions: () => Promise.resolve({ behavior:
'allow' })
```

Allow by default, but overridable. User permission settings, the project's .claude/settings.json, and the hooks system stack into three layers of control.

isDestructive() is the most important one. Tools that return true - deleting files, force-pushing, resetting databases - trigger extra confirmation. The system prompt repeats the phrase "measure twice, cut once" over and over.

9. Ink: A Custom React Terminal Rendering Engine

Source location: src/ink/ (50+ files)

Claude Code's terminal UI isn't stitched together with console.log. It's built on Ink, a framework that renders React components into terminal ANSI escape sequences.

The layout engine uses Yoga, Facebook's Flexbox implementation. Components include Box, Text, Button, ScrollBox, AlternateScreen, Link, RawAnsi, Spacer, and Newline.

Performance optimizations stack on top of each other:

- Node cache (node-cache.ts)
- Line width cache (line-width-cache.ts)
- Render optimizer (optimizer.ts)
- Focus management system

- Bidirectional text support (bidi.ts) for right-to-left rendering in Arabic and Hebrew

Key parsing in parse-keypress.ts translates raw terminal escape sequences into structured events. There is hyperlink detection, so if your terminal supports it, links in Claude Code output are clickable. There is mouse-event hit testing too: click somewhere, and the system computes which component you clicked.

This is a graphical application running inside a terminal.

---

10. The 200-Line Truncation in the Memory System

Source location: src/memdir/

Claude Code reads a MEMORY.md file in the project. But it comes with limits:

- Maximum 200 lines
- Maximum 25 KB

Anything beyond that gets truncated. MEMORY.md is an index, not storage. Each memory is its own markdown file, and MEMORY.md only holds one-line pointers.

There are four memory types: user, feedback, project, reference.

The system automatically searches for relevant memories with isAutoMemoryEnabled() and prefetches them asynchronously. In coordinator mode, there is a separate team memory path in teamMemPaths.ts, so workers can share memories.

The design philosophy of the memory system is: remember non-obvious things. You can read code structure from the code. You can get git history from git log. Those don't need memory. What does need memory is "last time the user said not to use try-catch" or "deployment is frozen until March 5."

---

11. Hooks System: 25 Lifecycle Events

Source location: src/utils/hooks.ts

Claude Code's extensibility doesn't come from an app store. It comes from shell hooks. There are 25 lifecycle events:

- Session-level: session_start, session_end
- Tool-level: pre_tool_use, post_tool_use, post_tool_use_failure
- Compaction-level: pre_compact, post_compact
- Sub-agent level: subagent_start, subagent_stop
- Task-level: task_created, task_completed
- Config-level: config_changed
- File-level: file_changed
- Permission-level: permission_denied

Each event can trigger a shell command. The command can return

JSON for structured decisions, asynchronous JSON for long-running operations, or a prompt request that asks the user for input.

That means you can do things like this: every time Claude wants to execute rm, run a validation script first; every time it creates a file, automatically add a license header; every time a session ends, send a Slack message.

Hooks are Claude Code's most underrated feature. All those long rules in the system prompt - "don't push to main," "don't delete branches" - can be enforced with hooks instead of relying on AI self-discipline.

---

12. moreright: A Directory With Only a Shell

Source location: src/moreright/useMoreRight.tsx

This directory contains only one file. It exports three empty functions: onBeforeQuery, onTurnComplete, and render.

The comment says "stub for external builds." The real implementation is for Anthropic's internal use only.

The name means nothing, just like the tengu_ feature flags. It might be a UI enhancement, or a monitoring system, or an experiment framework. The source offers no more clues.

---

13. Bridge: 32 Parallel Cloud Sessions

Source location: src/bridge/

Bridge is Claude Code's cloud session manager. The main file, bridgeMain.ts, is over 115 KB.

Backoff strategy:

- Connection failures: start at 2 seconds, cap at 120 seconds, give up after 10 minutes
- General errors: start at 500 ms, cap at 30 seconds, give up after 10 minutes
- Shutdown grace period: 30 seconds

Default parallel session count: 32. Status update frequency: every 1000 milliseconds.

It supports git worktree isolation. Sub-agents work in separate git worktrees and get merged back afterward. If a sub-agent didn't change anything, the worktree is cleaned up automatically.

JWT tokens refresh on a timer. It also supports "trusted device tokens," a hardware-level authentication mechanism.

The existence of Bridge means Claude Code is not just a local CLI tool. It can push compute-heavy work to the cloud while the local machine only coordinates and renders. And 32 parallel sessions means that in coordinator mode, you can have 32 workers running at once.

---

14. The Cache Boundary in the System Prompt

Source location: src/constants/system.ts

```
export const SYSTEM_PROMPT_DYNAMIC_BOUNDARY =
  '__SYSTEM_PROMPT_DYNAMIC_BOUNDARY__'
```

The system prompt is split into two halves by this marker.

Everything before the marker is cacheable across organizations: safety rules, tool definitions, output format. It's the same for all users. Anthropic's API layer can cache that part and avoid processing it again.

Everything after the marker is user-specific: contents of CLAUDE.md, git status, current date, session state. That part cannot be cached, because it changes every request.

There is also an explicit cache-break mechanism:

```
DANGEROUS_uncachedSystemPromptSection()
```

The function name starts with DANGEROUS_. Anything wrapped with it is recomputed every time and never cached. It's used for naturally dynamic content such as git status, which can change constantly, and the current date, which changes daily.

This is cost engineering. Prompt Cache reads cost 10% of writes. If 80% of the system prompt can be cached, each request saves 80% x 90% = 72% of the system-prompt processing cost. At million-user scale, that's worth millions of dollars.

---

15. Dreaming

Source location: src/services/autoDream/

There is literally a system in the code called Dream.

The comment in autoDream.ts is direct:

```
Background memory consolidation. Fires the /dream
prompt as a forked
subagent when time-gate passes AND enough sessions
have accumulated.
```

The trigger conditions: more than 24 hours since the last consolidation, and at least 5 accumulated sessions. Once those conditions are met, Claude Code launches a background sub-agent to perform a "dream" task.

The dream prompt has four phases:

1. **Orient**: read the existing memory files and understand the current state
2. **Gather recent signal**: search session transcripts for information worth remembering
3. **Consolidate**: merge new information into existing memories, correct outdated facts, convert relative dates into absolute dates
4. **Prune and index**: remove contradictory memories and keep the index under 200 lines

This system imitates human memory consolidation. During the day you experience events. During sleep the brain reorganizes memory, discards useless information, and strengthens important patterns. Claude Code does the same thing. You use it all day, and at night it "dreams" in the background, turning scattered sessions into structured memory.

The feature flag is called tengu_onyx_plover. Plover is a shorebird.

---

16. Anti-ptrace Defense: Preventing GDB From Reading Memory

Source location: src/upstreamproxy/upstreamproxy.ts

Inside the CCR (Cloud Code Runtime) container, Claude Code calls a Linux system call on startup:

```
prctl(PR_SET_DUMPABLE, 0)
```

This is done through Bun's FFI, directly calling libc's prctl function. Setting PR_SET_DUMPABLE to 0 means other processes running under the same user can no longer ptrace this process.

Why? Because Claude Code's memory holds session tokens. If someone uses prompt injection to get Claude to run gdb -p $PPID, GDB can attach to the parent process and read the token straight out of heap memory.

prctl(PR_SET_DUMPABLE, 0) closes that path.

The code goes further:

- It reads the token from /run/ccr/session_token
- After reading it, it deletes the token file, so the token exists only in heap memory and no longer on disk
- The timing is careful: it must delete the file only after the relay starts successfully. If it deletes early and the relay fails, the supervisor won't be able to read the token on restart

Most web developers would never think to call prctl. Anthropic's engineering team is defending against prompt-injection attacks that can make the AI execute arbitrary shell commands.

---

17. The Circuit Breaker for Context Compaction

Source location: src/services/compact/autoCompact.ts

When a Claude Code conversation exceeds the context window, it automatically triggers "compaction," summarizing older conversation history into a condensed version. This mechanism is called autoCompact.

But compaction sometimes fails. A BQ dashboard on March 10, 2026 showed that 1,279 sessions had more than 50 consecutive compaction failures, and the worst session failed 3,272 times.

Each failure is an API call. 1,279 sessions times hundreds of retries on average works out to roughly 250,000 wasted API calls per day.

The fix is a circuit breaker:

```
const MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES = 3
```

After 3 consecutive failures, retries stop. The code comment is unusually candid:

```
// BQ 2026-03-10: 1,279 sessions had 50+ consecutive
failures
// (up to 3,272) in a single session, wasting ~250K
API calls/day globally.
```

The compaction system itself has layers. First it tries Session Memory Compaction, replacing full conversation history with existing session memory. If that isn't enough, it falls back to Legacy Compaction, launching a sub-agent to read the full conversation and write a summary.

The Legacy Compaction prompt asks for 9 sections: main request, technical concepts, files and code snippets, errors and fixes, problem solving, all user messages, outstanding tasks, current work, next steps. It also has an <analysis> tag that acts as scratch paper. After the summary is produced, the analysis is stripped out and never enters context.

Claude Code spends tokens on analysis, then throws the analysis away and keeps only the summary. Because the analysis helps the model produce a better summary. It's a variant of chain-of-thought.

---

18. Auto Mode Classifier: Let Another Claude Decide Whether to Allow It

Source location: src/utils/permissions/yoloClassifier.ts

Claude Code's Auto Mode has a classifier: another Claude instance whose only job is to decide whether an operation is safe.

The classifier's system prompt lives in its own .txt file, auto_mode_system_prompt.txt. In external builds, dead-code elimination stripped out the contents, so we can't see the full prompt.

But the code structure makes a few things clear:

- There are three rule types: allow, soft_deny, and environment
- Users can customize all three and override the defaults
- Anthropic employees use a different permissions template, permissions_anthropic.txt, while external users use permissions_external.txt

Dangerous commands have a hardcoded blacklist:

```
const DANGEROUS_BASH_PATTERNS = [
  'python', 'python3', 'node', 'deno', 'tsx', 'ruby',
'perl',
  'php', 'lua', 'npx', 'bunx', 'bash', 'sh', 'ssh',
  'zsh', 'fish', 'eval', 'exec', 'env', 'xargs',
'sudo',
]
```

These commands can execute arbitrary code, so Auto Mode never auto-allows them.

The blacklist for Anthropic employees is longer. It also includes curl, wget, gh, git, kubectl, aws, gcloud. The code comments say those choices come from "ant sandbox data" - observed experience inside Anthropic - not from a universal theory that these tools are always unsafe.

There is also a denial-tracking system in denialTracking.ts: if the classifier denies 3 times in a row, or 20 times total, the system falls back to asking the user directly. The classifier is imperfect. Better to ask the user than keep rejecting legitimate operations forever.

---

19. Diminishing-Returns Detection for Token Budgets

Source location: src/query/tokenBudget.ts

When you give Claude Code a token budget, maxBudgetUsd, the system doesn't wait until the budget is fully spent before stopping. It has a diminishing-returns detector:

```
const isDiminishing =
  tracker.continuationCount >= 3 &&
  deltaSinceLastCheck < DIMINISHING_THRESHOLD &&
  tracker.lastDeltaTokens < DIMINISHING_THRESHOLD
```

DIMINISHING_THRESHOLD is 500 tokens. If, for 3 or more consecutive rounds, each round adds fewer than 500 tokens, the system decides the model is spinning its wheels: producing less and less, maybe repeating itself or hesitating.

At that point it stops early, even if only 60% of the budget is gone. No point spending money on a loop that isn't making progress.

There is also a 90% completion threshold. Until 90% of the budget is

There is also a 90% completion threshold. Until 90% of the budget is consumed, the system sends a nudge telling the model to keep pushing toward completion.

---

20. caffeinate: Preventing a Mac From Sleeping While It Works

Source location: src/services/preventSleep.ts

When Claude Code runs a long task, it calls macOS's caffeinate command to keep the machine awake:

```
spawn('caffeinate', ['-i', '-t', '300'], { stdio:
'ignore' })
```

-i creates an assertion preventing idle sleep. -t 300 sets a 5-minute timeout. That's a self-healing mechanism: if the Node process is killed with SIGKILL and cleanup handlers never run, the orphaned caffeinate process exits on its own after 5 minutes.

The system restarts caffeinate every 4 minutes, before the 5-minute timeout expires, so the wake lock stays continuous.

There is also a reference-counting mechanism for nested calls. Multiple sub-agents can ask to keep the system awake at the same time, and sleep is allowed only when the last one finishes.

Easy detail to miss.

---

21. Fennec: Opus 4.6 Used to Have a Codename

Source location: src/migrations/migrateFennecToOpus.ts

The migration scripts reveal Anthropic's internal codename system. Before release, Opus 4.6's internal codename was **Fennec**.

```
if (model.startsWith('fennec-latest[1m]')) {
  updateSettingsForSource('userSettings', { model:
'opus[1m]' })
} else if (model.startsWith('fennec-latest')) {
  updateSettingsForSource('userSettings', { model:
'opus' })
} else if (model.startsWith('fennec-fast-latest')) {
  updateSettingsForSource('userSettings', { model:
'opus[1m]', fastMode: true })
}
```

This migration runs only for USER_TYPE === 'ant', meaning Anthropic employees. External users never saw the name fennec. It was a purely internal codename.

Other migration scripts reveal the product's evolution:

- migrateSonnet1mToSonnet45.ts - Sonnet 1M got renamed to Sonnet 4.5
- migrateSonnet45ToSonnet46.ts - then upgraded again to Sonnet 4.6
- migrateOpusToOpus1m.ts - Max and Team Premium users got silently moved to the 1M-window Opus
- resetProToOpusDefault.ts - Pro users were reset to the default Opus

---

22. Context Collapse: Not Shipped Yet

Source location: multiple references to the CONTEXT_COLLAPSE feature flag

The code references CONTEXT_COLLAPSE and marble_origami all over the place. This is an experimental feature called context collapse, a more granular context-management system than autoCompact.

marble_origami is an internal agent, a ctx-agent, dedicated to context management. When its own context window is about to overflow, autoCompact is disabled, because marble_origami's resetContextCollapse() would corrupt the main thread's committed log, which lives in module-level shared state.

Sixteen files reference this feature. It has its own visualization component, ContextVisualization.tsx, its own command, /context, and its own analysis module, analyzeContext.ts.

The code comments expose the conflict between autoCompact and context collapse:

```
// Autocompact firing at effective-13k (~93% of
effective) sits right
// between collapse's commit-start (90%) and blocking
(95%), so it
// would race collapse and usually win, nuking
granular context that
// collapse was about to save.
```

The two systems are competing for the same context space. autoCompact fires at 93%. Context collapse starts committing at 90%. If both are enabled, autoCompact usually wins and wipes out the granular context that collapse was about to preserve.

The solution is simple: when context collapse is enabled, disable autoCompact. The two systems cannot coexist.

---

23. Speculative Execution Engine

Source location: src/services/PromptSuggestion/speculation.ts (992 lines)

Claude Code has a speculative execution engine. After you press Enter, the system predicts what you'll do next and starts executing before you type it.

How it works:

1. After Claude finishes a response, the system generates a guess for "what you might say next"
2. It immediately launches a sub-agent to act on that prediction
3. File writes go through a copy-on-write overlay, so writes land in /tmp/claude/speculation/{pid}/{id}/ instead of touching your real files
4. If you actually say that thing, or accept the suggestion, the overlay gets copied back into the main directory and the completed work is injected into the conversation
5. If you say something else, the overlay is silently deleted and it is as if nothing happened

Safety boundaries:

- If it encounters file editing, stop, unless you're in acceptEdits mode
- If it encounters a non-read-only Bash command, stop
- If it encounters an unknown tool, stop
- Maximum 20 rounds or 100 messages

Anthropic employees see feedback like: "Speculated 3 tool uses · 2.5K tokens · +1m23s saved."

This is the AI version of CPU speculative execution.

CPUs predict branch direction and execute instructions early. Claude Code predicts user intent and executes tool calls early. If it guesses right, time is saved. If it guesses wrong, the work is discarded and the user never notices.

There is also a pipeline mechanism: when one speculative execution finishes, the system immediately predicts the next next step and queues another speculation. One speculation ends, the next one is already lined up.

---

24. Anger Detector

Source location: src/utils/userPromptKeywords.ts

The entire file is 27 lines long, and all it does is detect whether the user is cursing.

```
const negativePattern =
  /\b(wtf|wth|ffs|omfg|shit(ty|tiest)?|dumbass|
horrible|awful|
  piss(ed|ing)? off|piece of (shit|crap|junk)|what
the (fuck|hell)|
  fucking? (broken|useless|terrible|awful|horrible)|
fuck you|
  screw (this|you)|so frustrating|this sucks|damn
it)\b/
```

wtf, piece of shit, fuck you, this sucks - all matched by regex.

What happens when it's triggered? A feedback survey appears.

---

25. Commit Attribution: Tracking Who Wrote the Code at Character Level

Source location: src/utils/commitAttribution.ts (references src/utils/attribution.ts)

Claude Code calculates the ratio of human and AI contribution for each commit. Not by file count. Not by line count. By character.

It uses a common prefix/suffix matching algorithm to determine, character by character, whether each piece was written by a human or by AI. There are 962 lines of code dedicated to this one job.

There is also a function called isUndercover(). If an Anthropic employee enables undercover mode, the commit contains no AI attribution at all. Anthropic employees can quietly use Claude Code without leaving traces in commit metadata.

Model codename leak prevention lives here too. The code comment says:

```
// @[MODEL LAUNCH]: Update the hardcoded fallback
model name below
// (guards against codename leaks).
```

If the user is on an unreleased internal model like fennec, the attribution system falls back to Claude Opus 4.6, preventing internal codenames from leaking into public repositories through commit messages.

---

26. /btw: Shower Thoughts

Source location: src/commands/btw/btw.tsx

/btw is a hidden command meaning "by the way." It launches a sub-agent to answer a question unrelated to the current task, like something you suddenly think of in the shower but don't want to let derail the current workflow.

The sub-agent uses runSideQuestion(), running in an isolated context without affecting the main conversation. The answer appears in a small scrollable window.

---

27. /think-back: A Year in Review

Source location: src/commands/thinkback/thinkback.tsx

Claude Code has a year-in-review feature. It loads from a marketplace plugin, reads all your session transcripts, and generates a summary: what you did with Claude Code this year, how much code you wrote, how many problems you solved.

Spotify Wrapped, but for programmers.

---

28. 160 Loading Verbs

Source location: src/constants/spinnerVerbs.ts

Claude Code doesn't say "Loading..." while it works. It has a list of 160+ random verbs:

"Lollygagging...", "Beboppin'...", "Boondoggling...", "Flibbertigibbeting...", "Clauding...", "Prestidigitating...", "Razzle-dazzling...", "Discombobulating..."

Users can customize the list in settings. There are two modes: replace, which replaces the default list, and append. You can make it say "goofing off..." while it loads.

Clauding is in the list: it turns its own name into a verb.

---

29. Magic Docs: Documentation That Updates Itself

Source location: src/services/MagicDocs/magicDocs.ts

Write # MAGIC DOC: [title] at the top of any markdown file, and Claude Code will keep that file updated automatically in the background.

After each conversation ends, a sub-agent checks whether the conversation produced information worth adding to the document. New discoveries get merged in. Outdated facts get removed.

The documentation philosophy is written directly into the prompt:

```
- BE TERSE. High signal only. No filler words.
- Do NOT duplicate information obvious from reading
the source code
- Focus on: WHY things exist, HOW components connect
- Skip: detailed implementation steps
```

Users can add italicized text on the line below the title as custom instructions, for example *Only record API changes*. Custom prompts are supported too through ~/.claude/magic-docs/prompt.md.

This is a self-maintaining knowledge base. You don't need to remember to update the docs. The docs update themselves.

---

30. Fast Modes Internal Codename: Penguin

Source location: src/utils/fastMode.ts, src/utils/config.ts

Fast Mode's internal codename is **penguin**. The code has things like penguinModeOrgEnabled, a feature flag called tengu_penguins_off, and an API endpoint /api/claude_code_penguin_mode.

Fennec was the codename for Opus 4.6. Penguin is the codename for Fast Mode. Anthropic's engineering team names features after animals.

---

31. API Key Prefix: Assembled at Runtime

The full string sk-ant-api never appears directly in the code. It's assembled like this:

```
['sk', 'ant', 'api'].join('-')
```

CI/CD scans the compiled bundle for leaked internal strings. The full API-key prefix would trigger an alert. Splitting it into three parts and joining them at runtime gets past the check.

Buddy pet names use hex encoding for the same reason. The ASCII spelling of duck collided with a model codename and triggered the build checker. The workaround was String.fromCharCode(0x64,0x75,0x63,0x6b).

---

32. Session Slugs Hide Computer Scientists

Session identifiers use the format adjective-verb-noun, like fuzzy-dancing-capybara.

The noun list has 415 words. Among them are babbage, dijkstra, lovelace, turing, knuth, hopper, berners-lee.

Your session might be called crispy-debugging-turing.

---

33. Query Loop: A Six-Layer Context-Management Pipeline

Source location: src/query.ts (1200+ lines)

Claude Code's core is a while(true) query loop. Before an API request is sent, messages pass through six layers of processing:

```
raw messages
  -> applyToolResultBudget (persist oversized tool
results to disk)
  -> snipCompact (clip old intermediate messages)
  -> microCompact (compress individual tool-result
contents)
  -> contextCollapse (granular context collapse)
  -> autoCompact (full-conversation compaction)
  -> normalizeMessagesForAPI (format into API-
compatible messages)
```

Each layer runs independently. The order matters. microCompact runs before autoCompact, because it may free enough space that autoCompact no longer needs to fire. contextCollapse also runs before autoCompact, because if collapse gets usage below the threshold, autoCompact won't destroy the granular context.

autoCompact won't destroy the granular context.

There are seven reasons the loop may continue: tool calls need another round, max_output_tokens recovery, token budget not yet spent, retry after reactive compaction, overflow recovery after context collapse, and two error-recovery paths. Every time it continues, the full six-layer pipeline runs again.

---

34. The Wizard Comment

Source location: src/query.ts:151-163

There is a real comment in production code that says this:

```
/**
 * The rules of thinking are lengthy and fortuitous.
They require
 * plenty of thinking of most long duration and deep
meditation
 * for a wizard to wrap one's noggin around.
 *
 * Heed these rules well, young wizard. For they are
the rules of
 * thinking, and the rules of thinking are the rules
of the universe.
 * If ye does not heed these rules, ye will be
punished with an
 * entire day of debugging and hair pulling.
 */
```

That comment documents three rules for thinking blocks: they must be in requests where max_thinking_length > 0, they cannot be the final block, and they must be preserved through the entire tool-call chain.

The engineer who wrote it was clearly tortured by these rules. "An entire day of debugging and hair pulling" reads less like style and more like testimony.

---

35. Tool Concurrency Partitioner: Parallel Reads, Serial Writes

Source location: src/services/tools/toolOrchestration.ts

When Claude calls multiple tools in one response, the system does not execute them all serially. It uses a partitioner called partitionToolCalls:

- Concurrency-safe tools, isConcurrencySafe: true, like Read, Glob, and Grep, can run in parallel
- Non-concurrency-safe tools like Edit, Write, and Bash have side effects and must run serially

The default max concurrency is 10, controlled by the environment variable CLAUDE_CODE_MAX_TOOL_USE_CONCURRENCY.

There is also a streaming executor, StreamingToolExecutor. Tool execution begins while the API response is still streaming. The system

doesn't wait for the full assistant response to finish. As soon as a tool_use block arrives, it starts the tool immediately. Results are buffered and emitted in receive order.

If a Bash tool errors, the executor aborts all sibling child processes through siblingAbortController, but it does not abort the parent. The query loop continues and handles the error result.

---

36. Undercover Mode: How Anthropic Employees Contribute Anonymously to Public Repos

Source location: src/utils/undercover.ts

Anthropic employees have an "undercover mode." When they contribute code to public or open-source repositories, Claude Code automatically activates a set of protective measures.

The system prompt injects an emergency instruction:

```
## UNDERCOVER MODE — CRITICAL

You are operating UNDERCOVER in a PUBLIC/OPEN-SOURCE
repository.
Do not blow your cover.

NEVER include in commit messages or PR descriptions:
- Internal model codenames (animal names like
Capybara, Tengu, etc.)
- Unreleased model version numbers (e.g., opus-4-7,
sonnet-4-8)
- The phrase "Claude Code" or any mention that you
are an AI
- Co-Authored-By lines or any other attribution
```

There is no force-off option. The code comment says it plainly:

```
// There is NO force-OFF. This guards against model
codename leaks —
// if we're not confident we're in an internal repo,
we stay undercover.
```

Default behavior: unless the repository's remote URL is on an internal allowlist, undercover mode stays on. The safe default is "pretend to be human."

That means Anthropic engineers may be contributing code to open-source projects you use, with commit messages that look completely human and leave no visible AI trace.

---

37. Efficiency Constraints on the Memory Extraction Agent

Source location: src/services/extractMemories/prompts.ts

After each conversation round, Claude Code launches a memory-extraction agent, a sub-agent whose job is to pull out information worth remembering long-term.

This agent has strict efficiency constraints:

```
You have a limited turn budget. Edit requires a prior
Read of the
same file, so the efficient strategy is: turn 1 —
issue all Read
calls in parallel for every file you might update;
turn 2 — issue
all Write/Edit calls in parallel.
```

Everything must be done in two turns. First turn: read every file that might need updating, in parallel. Second turn: write every update, in parallel. No interleaving. Alternating reads and writes would waste turns.

Even stricter, it is forbidden to verify information:

```
You MUST only use content from the last ~N messages
to update your
persistent memories. Do not waste any turns
attempting to investigate
or verify that content further — no grepping source
files, no
reading code to confirm a pattern exists, no git
commands.
```

It is not allowed to check the codebase to confirm whether the user is correct. If the user said it, the memory agent records it. That's a tradeoff between efficiency and accuracy. Verification would cost extra tool calls and extra rounds, but memory extraction is a background task with a tight budget.

---

38. Thinking Block Signatures: Cryptographic Auth and Credential Binding

Source location: src/utils/messages.ts:5061-5081

Thinking blocks and redacted_thinking blocks carry cryptographic signatures. Those signatures are bound to the API key that produced them.

If the user runs /login and switches accounts, the old thinking-block signatures become invalid and the API returns a 400 error.

The fix is stripSignatureBlocks(), which deletes all signed blocks after credentials change. The price is losing the model's prior thought process, but the system avoids the API error.

This reveals something about Anthropic's security architecture: thinking blocks are not just internal reasoning text. They are cryptographically signed artifacts. You cannot forge one, and you cannot transplant one

signed artifacts. You cannot forge one, and you cannot transplant one account's thinking block into another account's conversation.

---

39. MicroCompact: It Doesnt Compact Every Tool

Source location: src/services/compact/microCompact.ts

MicroCompact doesn't treat every tool result the same way. It has a whitelist:

```
const COMPACTABLE_TOOLS = new Set([
  'Read', 'Bash', 'PowerShell', 'Grep', 'Glob',
  'WebSearch', 'WebFetch', 'Edit', 'Write',
])
```

Only results from these tools can be micro-compacted, replaced with [Old tool result content cleared]. Other tool results, such as Agent outputs, remain intact.

There is also a "cached microcompact" experiment, CACHED_MICROCOMPACT, for Anthropic internal use only. Instead of deleting content, it edits the API prompt cache and tells the cache layer that certain tokens were removed while preserving the cached prefix. That avoids invalidating the whole cache when old results are cleared.

---

40. The Longest Type Name in the World

Source location: src/services/analytics/metadata.ts:57

```
export type
AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILE
PATHS = never
```

That type name is 56 characters long. Its value is never, which means it can never hold any value at all.

It exists for one reason only: whenever you send a string into analytics, you have to cast it with as AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS. That forces every engineer writing telemetry code to literally type out "I VERIFIED THIS IS NOT CODE OR FILEPATHS."

You cannot finish typing that cast and still honestly claim you "accidentally" sent a user's code snippet. The type name is a human speed bump. A forced pause.

There are hundreds of uses of this type across the codebase. Each one is an explicit declaration by an engineer: "I checked."

---

41. Learn by Doing Mode: AI Deliberately Refuses to Write Code

Source location: src/constants/outputStyles.ts:68

Source location: src/constants/outputStyles.ts:83

Claude Code has a teaching mode where Claude deliberately refuses to write code.

```
ask the human to contribute 2-10 line code pieces
when generating 20+ lines involving:
- Design decisions (error handling, data structures)
- Business logic with multiple valid approaches
- Key algorithms or interface definitions
```

It inserts TODO(human) markers into the code and then gives you a "Learn by Doing" task card:

```
◉ Learn by Doing
Context: [what's built and why]
Your Task: [specific function to implement]
Guidance: [trade-offs to consider]
```

Then it stops. It doesn't output the full solution. It waits for you to write the code before continuing.

A teaching mode for an AI coding tool that refuses to code. That's precise product design. For students and junior engineers, the act of writing the code is the learning. Giving the answer directly is less useful than forcing them to do part of it themselves.

---

42. PowerShell Security as a Whack-a-Mole Game

Source location: src/tools/PowerShellTool/pathValidation.ts:74

```
// hasUnvalidatablePathArg → ask. This ends the
KNOWN_SWITCH_PARAMS
// whack-a-mole where every missing switch caused the
unknown-param
// heuristic to swallow the next arg (potentially the
positional path).
```

PowerShell argument validation became a whack-a-mole game. Every time the engineers discovered a new switch parameter missing from the allowlist, the heuristic would swallow the next argument - which might be a file path - letting path validation be bypassed.

They kept adding switches to the allowlist one by one. Each time they added one, a new missing one popped up. Eventually they gave up on enumerating them all and switched to a fallback rule: if the call contains anything we don't fully understand, ask the user.

That comment is a surrender letter from an engineer who lost a long war against PowerShell.

---

43. There Are 120 Computer Scientists Hidden in the Slugs

Source location: src/utils/words.ts

The noun list for session slugs isn't just cute animals and food. Between acorn, waffle, and unicorn, it hides 120 computer scientists:

Babbage, Dijkstra, Hopper, Knuth, Lovelace, Turing are the obvious classics. But the list also includes:

- **Eich**, Brendan Eich, creator of JavaScript
- **Torvalds**, Linus Torvalds, creator of Linux
- **Hejlsberg**, Anders Hejlsberg, creator of TypeScript
- **Hickey**, Rich Hickey, creator of Clojure
- **Matsumoto**, Matz, creator of Ruby
- **Rossum**, Guido van Rossum, creator of Python
- **Stallman**, Richard Stallman, founder of GNU
- **Wozniak**, Steve Wozniak, Apple co-founder
- **Hinton**, **LeCun**, **Bengio**, the deep-learning trio

Your Claude Code session might be named fizzy-composing-dijkstra or squishy-recursive-knuth.

The adjective list also includes programming concepts: async, idempotent, memoized, polymorphic, curried, sharded.

So you might see idempotent-debugging-torvalds or lazy-refactored-stallman.

And kettle, the name used by the Buddy pet system, is in the noun list too.

---

44. The Archaeology of DO NOT Comments

There are a lot of "DO NOT" comments across the codebase. Every one is a gravestone for a previous incident:

```
// DO NOT ADD MORE STATE HERE — BE JUDICIOUS WITH
GLOBAL STATE
```

Someone added too much global state.

```
// DO NOT CHANGE THIS TO CUMULATIVE — It would mess
up aggregation
```

Someone changed it. The aggregation report broke.

```
// DO NOT set maxOutputTokens here. Fork piggybacks
on main thread
```

```
on main thread
```

Someone set it. The sub-agent and the main thread started fighting over token limits.

```
// DO NOT count Claude's autonomous codebase
exploration
```

Someone counted it. Telemetry got skewed.

```
// DO NOT push to remote repository unless user
explicitly asks
```

Someone pushed. The user was not happy.

Every "DO NOT" comment is a scar left by someone who already stepped on the landmine. They aren't style rules. They are lessons.

---

45. The YOLO Classifier

The safety classifier's file is called yoloClassifier.ts. YOLO = You Only Live Once.

Auto Mode lets Claude Code perform operations without asking the user first. The classifier that decides which operations are safe to YOLO through is called the YOLO classifier.

The code comments don't hide it either:

```
// Breaks the yoloClassifier → claudemd → filesystem
→ permissions cycle.
```

Those four modules formed a circular dependency. The fix was to add a cache in bootstrap/state.ts to break the cycle. The comment literally spells out the cycle path.

A security classifier named YOLO, requiring special handling because of a circular dependency, is code humor all by itself.

---

46. BashTool: A 10,894-Line Mini Operating System

Source location: src/tools/BashTool/ (18 files, 10,894 lines)

BashTool is not a thin wrapper around child_process.exec(). It is a full command-safety analysis system.

Command semantics understanding. commandSemantics.ts knows that different commands use exit codes differently. grep returning 1 is not an error: it just means no match. diff returning 1 is not an error: it

not an error, it just means no match. diff returning 1 is not an error, it just means the files differ. test returning 1 is not an error; it means the condition is false. If you treat every non-zero exit code as an error, Claude will keep trying to "fix" things that aren't broken.

Tree-sitter AST analysis. treeSitterAnalysis.ts uses a tree-sitter parser to analyze the Bash abstract syntax tree instead of using regexes. It can distinguish \;, which is an argument to find, from ;, which separates shell commands. It can detect whether $() command substitution and <() process substitution are inside quotes. It can analyze heredoc quoting boundaries.

The date trap. readOnlyValidation.ts has a special callback to examine positional arguments to the date command. If the positional argument does not start with +, for example date 0101120025, that is not querying the date. That is setting the system clock. A command that looks harmless changes category from read to write based only on positional-argument format.

The same issue for hostname. hostname -f is read, because it prints the FQDN. hostname myserver is write, because it sets the hostname. The only difference is whether there is a positional argument. The source uses a regex to hard-enforce "no positional argument" before auto-allowing it.

UNC-path WebDAV defense. On Windows, a UNC path like \\evil.com\share\file can trigger a WebDAV request and leak the user's NTLM hash to a remote server. BashTool checks for this pattern before every command runs. A Linux-first CLI tool still bothered to defend against a Windows-specific credential-leak vector.

The 23 dangerous capabilities of tput. tput init and tput reset can execute iprog from terminfo, which is arbitrary code. tput pfloc can program function keys to execute local strings. tput mc5 redirects output to a printer. The code enumerates 23 dangerous terminfo capability names and blocks them one by one.

---

47. VCR: A Tape Recorder for API Calls

Source location: src/services/vcr.ts

VCR stands for Video Cassette Recorder. This system records API requests and responses into "tapes" - fixture files - and replays them in tests.

How it works: hash the request body with SHA-1 and use the hash as the fixture filename. Next time the same request appears, the system reads the file instead of sending an API call.

```
// Error message when CI has no fixture:
`Fixture missing: ${filename}. Re-run tests with
VCR_RECORD=1,
 then commit the result.`
```

CI is not allowed to record new fixtures. It can only replay them. Developers have to record fixtures locally, then commit them to the repository. That keeps CI deterministic while preventing API keys from being exposed to the CI environment.

VCR also has a dehydration step. Before hashing, it replaces dynamic content in the request - UUIDs, timestamps, temporary directory paths - with placeholders, so logically identical inputs hash to the same fixture.

---

48. The Epitaph System for Destructive Commands

Source location: src/tools/BashTool/destructiveCommandWarning.ts

Every destructive command gets a short epitaph:

```
git reset --hard    -> "Note: may discard uncommitted
changes"
git push --force    -> "Note: may overwrite remote
history"
git clean -f        -> "Note: may permanently delete
untracked files"
rm -rf              -> "Note: may recursively force-
remove files"
DROP TABLE          -> "Note: may drop or truncate
database objects"
kubectl delete      -> "Note: may delete Kubernetes
resources"
terraform destroy   -> "Note: may destroy Terraform
infrastructure"
```

Notice the wording. It's all "may," not "will." Because git push --force doesn't always overwrite anything if the remote hasn't diverged. rm -rf temp/ might be deleting disposable temp files that don't matter.

These warnings do not affect permission logic. The comments explicitly say they are "purely informational." They appear only inside permission-confirmation dialogs to give the user one more hint. Read "Note: may destroy Terraform infrastructure" and most engineers will pause for one extra second.

---

49. Sandbox Excluded Commands: Not a Security Boundary

Source location: src/tools/BashTool/shouldUseSandbox.ts:19-20

```
// NOTE: excludedCommands is a user-facing
convenience feature,
// not a security boundary. It is not a security bug
to be able
// to bypass excludedCommands
```

Users can configure commands that should skip the sandbox. But the source is explicit: this is not a security boundary. Bypassing it does not count as a security vulnerability.

The real security control is the permission-prompt system. Sandbox exclusion is just a convenience feature. If you run bazel build all the time and don't want to confirm it over and over, you can exclude it

and don't want to confirm it over and over, you can exclude it.

This comment is rare. Most code does not proactively say "this feature is not a security boundary." Anthropic's engineers are preempting a specific misunderstanding: someone sees excludedCommands, assumes it is a hard defense line, and then files a "security report" when they bypass it.

---

50. fds -l Flag: An Unexpected Security Risk

Source location: src/tools/BashTool/readOnlyValidation.ts:77-78

```
// SECURITY: -l/--list-details EXCLUDED — internally
executes `ls`
// as subprocess (same pathway as --exec-batch). PATH
hijacking
// risk if malicious `ls` is on PATH.
```

The -l flag for fd, the file-finding tool, looks harmless. It just shows detailed file info. But internally it launches an ls subprocess. If an attacker puts a malicious ls earlier on the PATH, fd -l turns into code execution.

It uses the same code path as --exec-batch. A "read-only" flag becomes "execute" through PATH hijacking because fd treats ls as an external command to invoke.

BashTool's safety analysis has to go that deep. It can't just check whether the command name is on an allowlist. It has to understand the internal behavior of each flag on each command.

Seen Together

Fifty findings. From the pet system to the positional-argument trap in date, from wizard comments to UNC-path WebDAV defense.

What's really interesting is not any single finding. It's the picture they form together.

This is not an editor plugin. Anti-ptrace defenses, caffeinate sleep management, libc FFI calls, circuit breakers - these are operating-system and infrastructure design patterns. Claude Code's team is not building a command-line tool. They are building an operating system that runs on your computer.

It has its own lifecycle. Dreaming, memory consolidation, context-collapse experiments - Claude Code is not just waiting between turns. In the background it reorganizes memories, clears stale information, and prepares for the next conversation. This is not a stateless API call. It's a persistent system with memory and maintenance cycles.

The security engineering is much deeper than expected. The classifier is not a regex over commands. It's another Claude instance judging safety. Permission denial has a circuit breaker. Too many denials, and the system falls back to asking the user. Token files are deleted immediately after reading. Processes are made non-ptraceable. Every layer defends against a concrete attack scenario, not an abstract "best

lays defends against a concrete attack scenario, not an abstract "best practice."

The migration scripts are a product-history archive. Fennec -> Opus. Sonnet 1M -> Sonnet 4.5 -> Sonnet 4.6. Every rename left behind a migration script. Some of them only apply to internal employees, so external users never knew Opus 4.6 was once called Fennec. But the source doesn't lie.

Two context-management systems are competing. autoCompact fires at 93%. Context collapse starts at 90%. The two cannot coexist. One destroys the data the other is trying to preserve. That means Anthropic is developing two different context-management strategies in parallel and hasn't settled the winner yet.

The diminishing-returns detector reveals an economic principle. The system can stop before the budget runs out if output gains fall below 500 tokens for 3 straight rounds. That's not just a technical rule. It's economics. Every token has a cost, and when marginal output approaches zero, more spending is waste.

The pet-system salt is friend-2026-401. April 1, 2026. April Fool's Day. Opus 4.6's codename was Fennec. The dreaming feature flag is tengu_onyx_plover. The context-collapse agent is marble_origami.

A team that lets AI dream, builds an AI version of CPU speculative execution, calls prctl, names features after fennecs and penguins, says "Flibbertigibbeting" while loading, and hides computer scientists inside session IDs.

The deepest Easter egg in Claude Code's source is not a code snippet. It's the way this team thinks.

Aaron

Founder of ConnectOnion

April 1, 2026, Sydney

Sources:

- Claude Code npm package (@anthropic-ai/claude-code v2.1.88) - full TypeScript source
- /src/buddy/ - pet system: species, rarity, ASCII art
- /src/vim/ - Vim state-machine implementation
- /src/utils/modelCost.ts - model pricing constants
- /src/voice/voiceModeEnabled.ts - voice mode and kill switch
- /src/coordinator/coordinatorMode.ts - coordinator mode system prompt
- /src/constants/system.ts - system prompt architecture and cache boundary
- /src/Tool.ts - tool safety-classification metadata system
- /src/bridge/bridgeMain.ts - cloud session manager
- /src/ink/ - custom React terminal rendering engine
- /src/memdir/ - memory system and 200-line truncation
- /src/services/autoDream/ - AI dreaming and memory consolidation

- /src/upstreamproxy/upstreamproxy.ts - anti-ptrace defense and token handling
- /src/services/compact/autoCompact.ts - context-compaction circuit breaker
- /src/services/compact/sessionMemoryCompact.ts - session-memory compaction
- /src/services/compact/prompt.ts - compaction-summary prompt template
- /src/utils/permissions/yoloClassifier.ts - auto-mode classifier
- /src/utils/permissions/dangerousPatterns.ts - dangerous-command blacklist
- /src/utils/permissions/denialTracking.ts - denial tracking and circuit breaker
- /src/query/tokenBudget.ts - token budget and diminishing-returns detection
- /src/services/preventSleep.ts - macOS caffeinate sleep prevention
- /src/migrations/migrateFennecToOpus.ts - Fennec codename migration
- /src/services/awaySummary.ts - "while you were away" session summary
- /src/services/PromptSuggestion/speculation.ts - speculative execution engine with copy-on-write overlay
- /src/utils/userPromptKeywords.ts - anger-detection regex
- /src/utils/attribution.ts, /src/utils/commitAttribution.ts - character-level commit attribution
- /src/commands/btw/ - shower-thought side-question command
- /src/commands/thinkback/ - year-in-review feature
- /src/constants/spinnerVerbs.ts - 160+ loading verbs
- /src/services/MagicDocs/ - self-updating documentation system
- /src/utils/fastMode.ts - penguin mode, the fast-mode codename
- /src/query.ts - core query loop and six-layer context-management pipeline
- /src/services/tools/toolOrchestration.ts - tool-concurrency partitioner
- /src/services/tools/StreamingToolExecutor.ts - streaming tool execution
- /src/utils/undercover.ts - undercover mode for anonymous Anthropic contributions
- /src/services/extractMemories/ - memory-extraction agent
- /src/services/compact/microCompact.ts - micro-compaction and cache editing
- /src/services/compact/grouping.ts - API turn-grouping algorithm
- **cc.openonion.ai/fun-facts** - ConnectOnion's Claude Code fun-facts collection
- Claude Code Reverse-Engineering Report (1) - analysis of the context injection mechanism

Enjoyed this article?

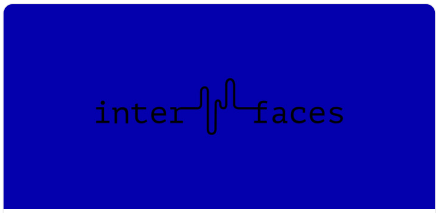
Follow to never miss an update.

Aaron xie

open source AI agent (ConnectOnion) /CEO of OpenOnion/machine learning engineer/ Blockchain Engineer/sci-fi writer/part-time researcher in medical area

Follow

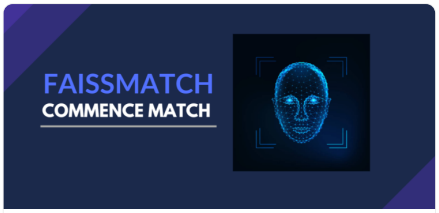
More articles for you



One use of Interface – Polymorphism C#

Jatin Nagpal

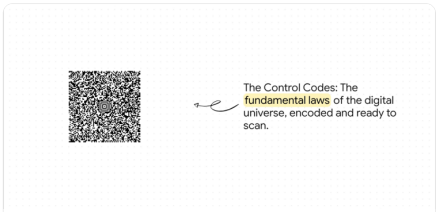
6 · 3 comments



Introducing FaissMatch to find similar faces from the gallery of images with the fastest search package(faiss).

Jaimin B.

27 · 5 comments



Aztec-Encoded PNG Artifacts

Brian Thorne

2 · 1 comment

Arc (C+

Ale›

2